

카드 모델 일체화 — 리팩터 설계도

`arm.card_ids: Array[String]` (ID 참조) → `arm.cards: Array[Dictionary]` (각 인스턴스가 카드 정의를 통째로 보유) 로 전환. `attack_bonus` 별도 필드를 제거하고 `arm_attack_boost` 를 카드 효과치 직접 변경으로 환원.

관련 원칙: `docs/범용적 원칙/자료구조_상향식_구현_방법론.md`, `docs/범용적 원칙/삼항 설계원칙.md` 관련 코드 시리즈: 직전 5 커밋 (주인공의 연구 — `d526c61` ~ `2ceaa6a`) 의 후속. 이번 리팩터로 `attack_bonus` 패턴 회수.

§0-pre. 왜 지금 하는가

현재 코드베이스에는 자료구조 일관성 균열이 있다:

- 팔의 `degradation` — 인스턴스 생성 시 `template.degradationduplicate(true)` 로 인스턴스에 박힘 (`run_manager.gd:99`). 이후 누구든 인스턴스의 `degradation` 을 건드리면 그 인스턴스에만 영향.
- 팔의 `card_ids` — 인스턴스에는 ID 문자열 배열만. 실제 카드 정의는 `GameData.CARD_TEMPLATES` 에 공유 상수로 존재. 카드 별 항구적 변경은 인스턴스 수준에서 못 함.

이 균열의 결과 — 직전 시리즈에서 `arm_attack_boost` 를 만들 때, 카드데미지를 직접 += 못 하고 별도 `attack_bonus` 필드를 도입해 `BattleManager` 가 계산 시점에 합산하는 구조가 강제됐다 (`battle_manager.gd:199` `(eff.value + bonus) * mult`). 이걸 카드 모델이 ID 참조라서 생긴 우회로지, 본질적으로 필요한 추상이 아니다.

카드도 `degradation` 과 동일하게 인스턴스에 박는 모델로 통일하면: - `arm_attack_boost` 가

`arm.cards[*].effects[*].value += amount` (`deal_damage` 인 경우만) 로 환원 — 보너스 필드·헬퍼 없음 - 미래의 카드 단위 강화 (편치별 자해 감소, 가드별 블록 보강 등) 가 같은 패턴으로 들어옴 - `punch_heavy_plus`, `punch_light_plus` 같은 변형 카드 ID 양산이 줄어들 — 하나의 카드를 여러 단계로 강화 가능

§0. 확정 사항

논의 거친 결과. 본 리팩터의 모든 코드는 아래 가정 위에서 작성.

- A. `card_id` 보존 (확정) — 각 카드 dict 가 `{id: "punch_light", name: ..., cost: ..., effects: [...], ...}` 형태. `id` 는 "현재 카드의 종류" 를 의미 (원본 계보 추적은 필요 시 별도 `origin_id` 필드). 비용 = 키 하나. 이득 = 디버그·로 그·향후 카드 단위 메커니즘·아트 매핑의 토대. `upgrade_card` 가 dict 교체 시 `id` 도 같이 갈아끼우는 책임 한 줄 추가.
- B. 인스턴스 생성 시 통째로 deep copy (확정) — `template.duplicate(true)`. Godot 의 Resource 참조 의미상 Texture2D / Shader / AudioStream 등은 자동으로 참조만 복사 (포인터). 데이터 값(숫자·문자·dict 구조) 만 deep copy. 카드 dict 안에 raw 바이트만 안 박으면 메모리 영향 0. 미래에 카드별 텍스처가 들어와도 동일.
- C. 전투 중 일시 변형 — 본 단계 밖 — 한 턴 데미지 +N 같은 일시 효과는 미도입. 도입 시 별도 설계. 다만 `BattleManager` 의 `arm_l_src.duplicate(true)` 패턴이 카드 dict 까지 deep copy 하므로, 향후 전투 중 카드 mutate 가 `RunManager` 의 arm 에 새지 않는 구조는 자동 확보.
- F. 카드별 `instance_id` 미도입 — 본 단계 밖 — 카드는 `arm.cards` 배열의 원소. 인덱스로 식별. 카드별 누적 상태 (사용 횟수, 강화 단계 추적 등) 도입 시 별도 설계.

자동 결정 — 논의 폐기:

- ~~D. 마이그레이션 정책~~ — 본 프로젝트는 디스크 세이브 미구현. `big_run_data` 가 메모리 전용. 리팩터 커밋 후 게임 재시작 → `_new_big_run()` 이 새 스키마로 빌드. 마이그레이션 코드 작성 안 함.
- ~~E. `attack_bonus` 제거 시점~~ — `attack_bonus` 는 본 리팩터의 의도된 회수 대상 (카드 모델 ID 참조 우회로). 같은 시리즈 마지막 단계에서 자연스럽게 제거.

§1. 목표

`arm.card_ids` 가 사라지고 `arm.cards` 만 남음. `attack_bonus` 필드와 `_arm_attack_bonus` 헬퍼도 사라짐. 게임 동작은 외부에서 보면 이전과 완전 동일.

완료 시점 검증 체크리스트:

- [] `git grep card_ids -- '*.gd'` 결과 0 건 (`game_data.gd` 의 템플릿 상수 `_DEFAULT_ARM_CARD_IDS` 는 보존)
- [] `git grep attack_bonus -- '*.gd'` 결과 0 건
- [] `git grep '(eff.value + bonus)' battle_manager.gd` 결과 0 건. `eff.value * mult` 형태로 환원

- [] 새 게임 → 노드 2 약편치 사용 → 적 HP -10 (보너스 0)
- [] 노드 4 진입 → 연구 옵션 중 `arm_attack_basic` 적용 → 회귀 후 노드 2 약편치 사용 → 적 HP -11
- [] `body_boost` / `arm_durability_boost` 동작은 직전 시리즈와 동일
- [] 전투 UI 의 핸드 카드가 직전과 동일 (이름·코스트·카테고리 색·affordable 표시)
- [] `upgrade_card` 콘솔 명령이 새 모델 위에서도 동작 (인덱스의 카드 dict 자체를 새 카드로 교체, id 도 같아끼움)

§2. 모듈 경계 — 삼항 설계원칙 적용

2-1. 변경 영향 영역

층	파일	변경 종류	변경 비중
상수	<code>game_data.gd</code>	무변. <code>CARD_TEMPLATES</code> 그대로, <code>_DEFAULT_ARM_CARD_IDS</code> 도 템플릿용으로 보 존	0
시물 (런)	<code>run_manager.gd</code>	<code>_create_arm_instance_in</code> / <code>end_internal_run</code> 역류 / <code>upgrade_card</code> / <code>_apply_arm_attack_boost</code>	4 곳
시물 (전투)	<code>battle_manager.gd</code>	<code>_build_initial_deck</code> / <code>_make_card_instance</code> / <code>play_card</code> / <code>_execute_card_effects</code> / <code>get_card_preview</code> / <code>_arm_attack_bonus</code> 제거	6 곳
UI	<code>battle_ui.gd</code>	<code>_make_card_view</code> / 핸드 빌드 시 disabled 체크	2 곳
UI	<code>run_ui.gd</code> / <code>research_ui.gd</code>	무변 (카드 직접 참조 없음)	0

2-2. 의존 방향 (변경 없음)

GameData ← RunManager ← (BattleManager) ← UI

`BattleManager` 가 `GameData.CARD_TEMPLATES` 록업을 그만두므로 의존 그래프 측면에서는 더 깨끗해짐 — `BattleManager` 는 이제 `card_inst.def` 만 읽고, 카드 정의의 출처는 `arm.cards` (`RunManager` 가 채워준 인풋) 뿐. **상수 직접 의존 한 줄 감소.**

2-3. 시물 자기완결성 검증

삼항 설계원칙 §4 "읽기 테스트": `game_data.gd` + `run_manager.gd` + `battle_manager.gd` 만 보면 카드가 어떻게 흐르는지 전부 답할 수 있어야 함. 본 리팩터는 이 가독성을 개선 — `card_inst.card_id` 로 록업하던 우회 한 단계가 사라지고 `card_inst.def` 직접 참조로 단순화.

2-4. 금지 항목 (불변)

- UI 가 카드 정의 mutate
- `BattleManager` 가 `RunManager` 의 `arm_instances` 직접 변경 (입력 페이로드 경우만)
- 카드 효과 로직이 UI 코드 안에 존재

§3. 자료구조 변경 — Before / After

3-1. `arm` 인스턴스

Before (`run_manager.gd`:91-100):

```
target["arm_instances"][id] = {
    "instance_id": id,
    "template_id": template_id,
    "name": template.name,
    "slot_type": template.slot_type,
    "max_hp": template.max_hp,
    "hp": template.max_hp,
    "card_ids": template.card_ids.duplicate(),      # ← 문자열 배열
    "degradation": template.degradation.duplicate(true),
    "attack_bonus": 0,                             # ← 본 작업에서 제거
}
```

After:

```
var cards: Array = []
for cid in template.card_ids:
    cards.append(_build_card(cid))

target["arm_instances"][id] = {
    "instance_id": id,
    "template_id": template_id,
    "name": template.name,
    "slot_type": template.slot_type,
    "max_hp": template.max_hp,
    "hp": template.max_hp,
    "cards": cards,                                # ← 카드 dict 배열
    "degradation": template.degradation.duplicate(true),
}

# 단일 카드 생성 - 카드 모델의 원자 단위.
# CARD_TEMPLATES 는 팔 외 다른 출처 (보상·이벤트·인벤토리 등) 에서도 호출될 예정이므로
# 함수 자체는 "팔" 맥락에 묶이지 않음. arm 의 카드 배열은 이 함수의 반복 호출 결과.
func _build_card(card_id: String) -> Dictionary:
    var def: Dictionary = GameData.CARD_TEMPLATES[card_id].duplicate(true)
    def["id"] = card_id                             # ← 결정 A: id 보존
    return def
```

각 카드 dict 의 스키마 (CARD_TEMPLATES 의 한 항목 + **id** 키):

```
{
    "id": "punch_light",                            # 결정 A
    "name": "약펀치",
    "cost": 1,
    "category": "attack",
    "effects": [
        { "type": "deal_damage", "value": 10 },    # ← 여기를 _apply_arm_attack_boost 가 직접 mutate
    ],
    "degradation_resistance": 0.1,
    "description": "",
}
```

3-2. **card_inst** (전투 중)

Before (battle_manager.gd:87-94):

```
func _make_card_instance(card_id: String, arm_side: String) -> Dictionary:
    var id: int = battle_state["next_card_instance_id"]
    battle_state["next_card_instance_id"] = id + 1
    return {
        "card_id": card_id,
        "arm_side": arm_side,
        "instance_id": id,
    }
```

After:

```
func _make_card_instance(card_def: Dictionary, arm_side: String) -> Dictionary:
    var id: int = battle_state["next_card_instance_id"]
    battle_state["next_card_instance_id"] = id + 1
    return {
        "def": card_def,
        "arm_side": arm_side,
        "instance_id": id,
    }
```

← 카드 정의 자체. arm 의 카드 dict 참조

deep copy 정책: `def` 는 `battle_state.arm_l.cards[i]` 의 원소 dict 를 직접 참조 (포인터). `begin_battle` 의 `arm_l_src.duplicate(true)` 가 이미 arm 통째로 deep copy 했으므로 cards 도 같이 deep copy 됨 → battle_state 안의 cards 는 RunManager 의 arm 과 격리. 따라서 `card_inst.def` 가 mutate 돼도 RunManager 의 arm 에 안 새어나감.

3-3. `card_id` 의 의미 이동

기존 `card_inst.card_id: String` → 제거. 대신 필요할 때 `card_inst.def.id: String` 로 접근.

호출처 변경: - `card_inst.card_id` 직접 사용 → `card_inst.def.id` (디버그·로그·향후 사용처) -
`GameData.CARD_TEMPLATES[card_inst.card_id]` 룩업 → `card_inst.def` 직접 사용

3-4. `attack_bonus` 제거

`arm_instances[*]["attack_bonus"]` 필드 삭제. `_arm_attack_bonus(side)` 헬퍼 삭제.

`_apply_arm_attack_boost` 재구현:

```
func _apply_arm_attack_boost(params: Dictionary) -> bool:
    var amount: int = params.get("amount", 1)
    for arm in big_run_data.get("arm_instances", {}).values():
        _bump_attack_cards(arm.get("cards", []), amount)
    for arm in run_data.get("arm_instances", {}).values():
        _bump_attack_cards(arm.get("cards", []), amount)
    return true

# 카드 배열 안의 deal_damage 효과들에 += amount.
func _bump_attack_cards(cards: Array, amount: int) -> void:
    for card in cards:
        for eff in card.get("effects", []):
            if eff.get("type", "") == "deal_damage":
                eff["value"] = int(eff.get("value", 0)) + amount
```

→ 이제 BattleManager 의 `deal_damage` 분기는 `eff.value * mult` 만 하면 됨. 보너스 합산 로직 사라짐.

3-5. 변경 없음 자료구조

- `body_max_hp` / `body_hp` — `_apply_body_boost` 그대로
- `arm.max_hp` / `hp` — `_apply_arm_durability_boost` 그대로
- `research_data` / `research_offers` / `research` 페이지 — 그대로
- `degradation` — 그대로 (이미 인스턴스에 박혀 있음)

§4. 의존 DAG (변경분만)

— 상수 —
GameData.CARD_TEMPLATES (leaf, 변경 없음)
↓ (인스턴스 생성 시점에 1회 deep copy)

— 시물 (런) —
arm.cards: Array[Dictionary] (NEW - 자기 안에 effects[*] 포함)
↑
arm_instances[id] (cards 필드 추가, card_ids / attack_bonus 제거)

— 시물 (전투) —
card_inst.def (NEW - arm.cards[i] 의 deep-copied 사본 참조)
↑
battle_state.deck / hand / discard 의 카드 인스턴스

— UI —
battle_ui (card_inst.def 직접 읽기 - GameData 룩업 사라짐)

검증: - 시물 → 상수 룩업 횟수 감소 ✓ (BattleManager 의 CARD_TEMPLATES 룩업 8 → 0) - 시물 → UI 역방향 — 없음 ✓ (불변) - 카드 정의의 단일 진실 — `arm.cards[i]` (인스턴스 단위) ✓ - 미래 카드 단위 변경의 단일 출처 = `arm.cards[i]` mutate

§5. 단계 분할

원칙: battle_manager 와 battle_ui 는 카드 정의 출처가 같으므로 한 커밋 안에서 함께 바뀌어야 함. attack_bonus 제거는 카드 모델이 새로 자리잡은 후 별도 커밋.

Step 1 — `arm.cards` 도입 + 모든 소비자 동기 (단일 커밋)

왜 한 커밋? `card_ids` 를 빼고 `cards` 만 두면, BattleManager 가 `arm.card_ids` 읽으려다 즉시 깨짐. 동시에 바뀌야 함.

파일: `run_manager.gd`, `battle_manager.gd`, `battle_ui.gd`

작업: 1. `run_manager.gd`: - `_build_card(card_id: String) -> Dictionary` 헬퍼 신설 (§3-1). 단일 카드 생성의 원자 단위 - `_create_arm_instance_in`: `card_ids` 키 → `cards` 키 (template.card_ids 를 순회하며 `_build_card` 호출). `attack_bonus` 키 잠시 보존 (Step 2 에서 제거) - `end_internal_run` 의 역류 루프: `card_ids` → `cards` - `upgrade_card(instance_id, index, new_card_id)`: 시그니처 보존하되 내부에서 `cards[index] = _build_card(new_card_id)` (한 줄) 2. `battle_manager.gd`: - `_build_initial_deck`: `for card_id in arm.card_ids` → `for card_def in arm.cards` - `_make_card_instance`: 시그니처 변경 (`card_def: Dictionary, arm_side: String`). 반환 dict 에 `def` 키 - `play_card`: `card_inst.card_id` 룩업 제거, `card_inst.def` 직접 사용 - `_execute_card_effects`: 인자는 `card_inst, card_def` 그대로 (호출자가 def 넘겨줌) - `get_card_preview`: 마찬가지로 `_arm_attack_bonus / (eff.value + bonus) * mult` — Step 2 까지 보존 (보너스 필드도 보존) 3. `battle_ui.gd`: - 두 군데 (`_make_card_view`, 핸드 disabled 체크) `GameData.CARD_TEMPLATES[inst.card_id]` → `inst.def` 직접 사용

검증: - 새 게임 → 콘솔 `show_run` 으로 `arm_instances[*].cards` 가 dict 배열 (각 카드의 `id`, `name`, `effects` 등 보임) 확인 - `card_ids` 키가 인스턴스에 없는지 확인 - 노드 2 전투 → 약편치 정상 작동, 적 HP -10 - 핸드 UI 의 카드 이름·코스트 정상 표시 - LimboConsole `upgrade_card 1 0 punch_heavy` → 0번 슬롯이 강편치로 교체. `show_run` 에서 `cards[0]` 의 `name` 이 "강편치", `id` 가 "punch_heavy" 확인 - 회귀 후 새 런에서도 카드 상태 보존

Step 2 — `attack_bonus` 제거 + `arm_attack_boost` 카드 직접 변경

파일: `run_manager.gd`, `battle_manager.gd`

작업: 1. `run_manager.gd`: - `_create_arm_instance_in` 의 `attack_bonus: 0` 줄 삭제 - `_apply_arm_attack_boost` 재구현 — `_bump_attack_cards` 헬퍼 신설. `cards[*].effects[*].value` 직접 += (deal_damage 효과만) 2. `battle_manager.gd`: - `_arm_attack_bonus(side)` 함수 삭제 - `_execute_card_effects` 의 `deal_damage` 분기: `(eff.value + bonus) * mult` → `eff.value * mult`. 코멘트 정리 - `get_card_preview` 의 동일 분기도 환원

검증: - 새 게임 → `show_run` 에 `attack_bonus` 키 없음 확인 - 노드 4 진입 → `arm_attack_basic` 적용 → `show_run` 에 `cards[*]` 중 약편치(`id: "punch_light"`) 의 `effects[0].value` 가 11 (10 + 1) 확인 - 회귀 후 노드 2 전투 → 약편치 사용 → 적 HP -11 -

(확정) 게임 재시작 시 잔여 `attack_bonus` 키 자동 사라짐 (새 `_new_big_run()` 호출이 새 스키마로 빌드하므로 마이그레이션 코드 불필요)

Step 3 — 검증 종합 + 정리

파일: 위와 동일, 변경 거의 없음

작업: 1. `git grep card_ids -- '*.gd' → 0` 건 2. `git grep attack_bonus -- '*.gd' → 0` 건 3. 디제틱 정체성 코멘트 정리 (`run_manager.gd` 의 `_apply_*` 함수들) 4. 콘솔 `show_run` 출력 한 번 캡처 (디버그용 코멘트로 첨부 가능)

§6. 테스트 시나리오

6-1. 단위 검증 (콘솔)

각 단계 끝나고 LimboConsole 으로:

```
new_game
show_run
# arm_instances[1].cards 가 6개 dict, 각 dict 에 id/name/effects 보이는지
# arm_instances[1].card_ids 가 없는지
# (Step 2 후) arm_instances[1].attack_bonus 가 없는지
```

6-2. 동작 회귀 검증

직전 시리즈 (주인공의 연구) 의 검증 시나리오 그대로 다시 돌아야 함:

1. 새 게임 → 노드 2 약펀치 사용 → 적 HP -10 (보너스 0)
2. 노드 2~3 승리 → 잔액 25
3. 노드 4 진입 → 연구 화면 표시
4. `arm_attack_basic` 적용 → "적용됨" + 잔액 -15
5. 종료 → 회귀, 잔액 10
6. 새 런에서 노드 2 약펀치 → 적 HP -11
7. 다시 노드 4 진입 → 다른 옵션 (예: `arm_durability_basic`) 적용 → 회귀
8. 다음 런에서 팔 `max_hp` +20 확인

6-3. UI 회귀 검증

- 핸드 카드 비주얼 (이름·코스트·카테고리 색·affordable 표시) 직전 commit 과 동일
- 호버 애니메이션 정상
- 카드 사용 → 데미지 프리뷰가 실제 데미지와 일치
- 카드 사용 후 discard 로 이동, hand 재구성 정상

6-4. 부정 시나리오

- `upgrade_card` 로 존재하지 않는 카드 ID 시도 → false (KeyError 방지)
- `_apply_arm_attack_boost` 적용 시 `cards` 배열이 빈 팔이 있는 경우 → 통과 (변경 0)
- 연구 효과 적용 후 회귀 안 한 상태에서 전투 → `run_data` 측 `cards` 가 변경됐으므로 즉시 데미지 반영 (실제로 일어나지 않는 시나리오지만 안전성 확인)

§7. 자가점검

7-1. 삼항 설계원칙

읽기 테스트 — 리팩터 후 `game_data.gd` + `run_manager.gd` + `battle_manager.gd` 만 보고 다음에 답할 수 있는가: - [] 카드는 어디에 저장되는가? (`arm.cards`) - [] 카드 효과 변경은 어디서? (`_apply_arm_attack_boost` 의 `_bump_attack_cards`) - [] BattleManager 가 카드 정의를 어디서 얻는가? (`card_inst.def` , `arm.cards` 에서 발원) - [] CARD_TEMPLATES 는 누가 읽는가? (RunManager 의 `_build_card` 만 — 1 곳)

교체 테스트: `battle_ui.gd` 삭제 → BattleManager·RunManager 컴파일 통과.

삭제 테스트: `_arm_attack_bonus` / `attack_bonus` 키워드 검색 0 건.

7-2. 자료구조 상황식

각 자료구조의 본 단계 안 사이클:

자료구조	생성	변경	조회
<code>arm.cards</code>	Step 1 (<code>_build_card</code> 반복 호출)	Step 2 (<code>_apply_arm_attack_boost</code>), <code>upgrade_card</code>	Step 1 (BattleManager <code>_build_initial_deck</code>)
<code>card_inst.def</code>	Step 1 (<code>_make_card_instance</code>)	(전투 격리, mutate 안 함)	Step 1 (<code>_execute_card_effects</code> , <code>get_card_preview</code> , <code>_make_card_view</code>)
<code>attack_bonus</code>	(제거 대상)	Step 2 에서 제거	Step 2 에서 제거

리프부터 위로 (CARD_TEMPLATES → arm.cards → card_inst.def → UI) 의 순방향 지킴.

7-3. 의존 그래프 정리 효과

측면	Before	After
BattleManager → GameData 록업	8 곳	0 곳
카드 정의 단일 진실	분산 (CARD_TEMPLATES 상수 + attack_bonus 보너스)	<code>arm.cards[*]</code> 단일
<code>_arm_attack_bonus</code> 헬퍼	존재 (보너스 합산 6 줄)	제거
카드 변형 도입 비용 (예: 자해 감소)	새 카드 ID 정의 필요	인스턴스에서 직접 mutate

§8. 커밋 단위

3 커밋. Step 1 이 가장 묵직, Step 2~3 은 cleanup.

커밋 1 — `arm.cards` 모델 도입

카드 모델 일체화 — `arm.cards` 도입 + 소비자 동기

- arm 인스턴스에 `cards: Array[Dictionary]` 추가, `card_ids` 제거
각 카드는 `CARD_TEMPLATES.duplicate(true) + id` 키 보존
- `_build_card` 헬퍼 (단일 카드 원자 단위), `end_internal_run` 역류 / `upgrade_card` 동기
- BattleManager: `card_inst` 가 `def` 를 직접 보유. `CARD_TEMPLATES` 록업 8곳 제거
- battle_ui: `card_def` 록업 2곳을 `inst.def` 직접 참조로 환원
- `attack_bonus` 필드는 본 커밋에선 보존 (다음 커밋에서 제거)

커밋 2 — `attack_bonus` 제거 + `arm_attack_boost` 환원

카드 모델 일체화 — `attack_bonus` 제거, `arm_attack_boost` 카드 직접 변경

- `_apply_arm_attack_boost` 가 `arm.cards[*].effects[*]` 의 `deal_damage` value 를 직접 += . `_bump_attack_cards` 헬퍼
- `_create_arm_instance_in` 의 `attack_bonus: 0` 줄 삭제
- BattleManager: `_arm_attack_bonus` 헬퍼 제거. `(eff.value + bonus) * mult` → `eff.value * mult` 로 환원. `get_card_preview` 도 동일

커밋 3 — 검증 마무리

카드 모델 일체화 — 검증·정리

- `git grep card_ids / attack_bonus` 0건 확인
- 코멘트 정리, 디제틱 여휘 일관성

§9. 다음 차로 (본 단계 바깥)

본 리팩터 마무리 후 자연스럽게 들어올 후속:

1. 카드 단위 강화 옵션 — `card_attack_boost` (특정 카드 한 장만 +damage), `card_cost_reduction` 같은 것. `arm.cards[i]` 직접 mutate 패턴 그대로
2. 카드 단위 `instance_id` — F2 도입 시 카드별 누적 상태 (사용 횟수, 강화 단계 추적 등) 가능
3. 자해 카드 자해량 변경 — 강펀치의 `damage_own_arm` 을 줄여주는 효과. 본 패턴 위에서 trivially 동작
4. 카드 변형 카드 — 본래의 `punch_heavy_plus` 식 변형은 더 이상 새 ID 양산 불필요. 기존 `punch_heavy` 의 effects 를 인스턴스 수준에서 변형 가능
5. 카드 인벤토리 (덱 빌딩) — 팔 외부에 보유 카드 풀. `arm.cards` 가 dict 단위라 자유롭게 옮김 가능
6. 세이브 / 로드 — `cards: Array[Dictionary]` 가 JSON 직렬화 친화적. 디스크 세이브 도입 시점에서야 마이그레이션 정책이 진짜 의미를 가짐 — 그때 `id` 필드가 핵심 매칭 키로 작동

§10. 자기검증 종합

- [x] 3층 분리 유지 §2-1 / 의존 한 방향 유지 §2-2 / 시물 자기완결 개선 §2-3 (CARD_TEMPLATES 록업 8 → 0)
- [x] 자료구조 변경 명세 §3 (Before/After) / DAG 변경분 §4
- [x] 단계 분할 §5 (Step 1 atomic, Step 2 cleanup, Step 3 verify)
- [x] 테스트 시나리오 §6 (단위 + 회귀 + UI + 부정)
- [x] 자료구조별 사이클 §7-2 / 의존 정리 효과 정량화 §7-3
- [x] 확정 사항 §0 (A · B 확정 / C · F 본 단계 밖 / D · E 자동 결정)

§0 확정 위에서 코드 진입 가능.